

Резюме кандидата

Василенко Максим Петрович

Позиция: Python разработчик

Уровень: Senior

Готов выйти на проект: ASAP

Локация: г. Минск, Беларусь

Сопроводительное письмо (О себе, знания и навыки)

Занимаюсь коммерческой разработкой на Python более 6 лет: асинхронный код (asyncio / FastAPI), строгая типизация, автотесты (pytest) и поставка backend-сервисов в production. Уверенный бэкэнд-фундамент: REST и gRPC, очереди и стримы (Kafka, Redis, RabbitMQ, Pub/Sub), PostgreSQL, Docker, базовая эксплуатация Kubernetes (EKS / AKS / Cloud Run + k8s). Есть практический опыт разработки ИИ-агентов и LLM-based решений в production: многошаговые агентские сценарии с вызовом инструментов, RAG (embeddings / retrieval / pgvector), пайплайны STT→LLM, guardrails и human-in-the-loop. ИИ-инструменты использую в ежедневной работе — от проектирования и код-ревью до ускорения разработки, анализа инцидентов и проверки гипотез по latency/стоимости LLM-вызовов.

Чек-лист

- + От 3 лет опыта в разработке на Python: асинхронный код, типизация, тесты
- + Уверенный бэкэнд-фундамент: REST/gRPC, очереди (Kafka / Redis / RabbitMQ), Postgres, Docker, базовая работа с k8s или аналогами
- + Практический опыт разработки ИИ-агентов и LLM-based решений
- + Использую ИИ-инструменты в ежедневной работе

Ключевые навыки (Основной стек)

Платформы	Linux, macOS, MS Windows
Языки программирования	Python (основной); практический опыт Golang на критичных участках
Инструменты	Python, asyncio, typing, FastAPI, Django, Django REST Framework, Flask, Pydantic, SQLAlchemy, Alembic, Celery, httpx, aiohttp, REST, gRPC, OpenAPI/Swagger, Kafka, Redis Streams / Pub-Sub, RabbitMQ, GCP Pub/Sub, Azure Service Bus, Docker, docker-compose, Kubernetes, Helm, ArgoCD, GitHub Actions, GitLab CI/CD, OpenAI API, Anthropic Claude, AI-агенты, RAG, embeddings, pgvector, STT→LLM пайплайны, guardrails, human-in-the-loop, Pytest, Prometheus, Grafana, Sentry, Git, Design patterns
Базы данных	PostgreSQL, Redis, MySQL, Cloud SQL, GCP Datastore; векторный поиск (pgvector)

Образование

Программное обеспечение информационных технологий, инженер-программист

Белорусский государственный университет информатики и радиоэлектроники (БГУИР), факультет компьютерных систем и сетей

Опыт работы

(Роль — Уровень — Компетенция)

Январь 2020 – Июль 2026

Senior Python разработчик. Backend и интеграции для высоконагруженных и регулируемых SaaS: микросервисы, event-driven, REST/gRPC, очереди, PostgreSQL, Docker/Kubernetes. Последние проекты — AI-агенты и LLM-based продукты в production (realtime-ассистент, RAG, antifraud-подсказки). Сильная сторона — надёжная доставка и наблюдаемость: идемпотентность, ретраи, back-pressure, аудит, тесты, разбор инцидентов.

Недавние проекты

Realtime AI-ассистент для страховых кол-центров (B2B SaaS)

Январь 2025 – Июль 2026

Набор использованных технологий: Python, Django, DRF, FastAPI, asyncio, PostgreSQL, pgvector, Redis, OpenAI API, Anthropic Claude, RAG, embeddings, GCP (Cloud Run, Pub/Sub, Cloud SQL, GCS), Kubernetes, Docker, GitLab CI/CD, Prometheus, Grafana, Pytest, Sentry

Описание проекта

Мультитенантный B2B SaaS: во время звонка система ведёт оператора по сценарию — транскрибация, извлечение намерений, подсказки, вызов инструментов и ответы с опорой на базу знаний. Критичны низкая задержка, изоляция клиентов и human-in-the-loop на рискованных решениях.

Что было сделано

- Отвечал за backend realtime-контура: приём аудио/событий звонка, статусы сессии, выдача подсказок оператору.
- Спроектировал мультитенантную архитектуру с изоляцией данных и прав доступа между клиентами платформы.
- Разрабатывал AI-агентские сценарии: многошаговые workflow, tool calling, память сессии, human-in-the-loop на рискованных шагах.
- Построил production RAG: ingest/chunking, embeddings, retrieval (pgvector), grounded-ответы LLM с опорой на источник.
- Внедрил пайплайны STT→LLM (OpenAI, Claude): промпты, guardrails, деградация без обрыва сценария звонка.

* Все поля обязательны для заполнения, изменение порядка полей не допускается.

- Разделил sync API и тяжёлые AI-стадии через GCP Pub/Sub: ретраи, back-pressure, correlation ID и auditable-статусы.
- Сравнивал варианты retrieval и LLM-вызовов по качеству, стоимости и latency; оставлял наблюдаемый контур.
- Оптимизировал задержку подсказок: кеш в Redis, контроль стоимости LLM-вызовов, устойчивость при сбоях внешних API.
- Проектировал схему PostgreSQL под сессии звонков, подсказки, retrieval-контекст и аудит действий оператора.
- Настроил CI/CD в GitLab CI/CD, Docker/Kubernetes, Prometheus/Grafana/Sentry; покрыл ключевые сценарии pytest.

Международный SaaS: страховые заявки и antifraud

Апрель 2024 – Декабрь 2024

Набор использованных технологий: Python, FastAPI, Django, DRF, PostgreSQL, Redis, Kafka, Airflow, Azure (Service Bus, Blob Storage, AKS), Pydantic, Alembic, Docker, Kubernetes, Pytest, OpenAPI

Описание проекта

B2B-платформа обработки страховых заявок и выявления мошенничества. Приём документов, извлечение данных, проверка кейса оператором, сигналы о подозрительных паттернах. Важны human review, партнёрские интеграции и надёжная доставка статусов.

Что было сделано

- Разработал backend обработки страховых заявок: загрузка документов, извлечение полей, статусы для проверки оператором.
- Реализовал antifraud-сигналы и AI-подсказки: скоринг/эвристики, оценка уверенности, переопределение человеком.
- Построил event-driven потоки на Kafka для статусов заявок и webhook-уведомлений партнёрам.
- Интегрировал внешних партнёров: версионизируемые контракты, идемпотентная доставка статусов.
- Оркестрировал пакетную обработку и ETL через Airflow; хранение вложений в Azure Blob Storage; контур AKS.
- Вынес тяжёлую обработку документов и antifraud-стадии в отдельные воркеры — меньше влияние сбоев на API.
- Участвовал в миграции Django → FastAPI на живом продукте без простоя; REST API с OpenAPI-спецификацией.
- Развивал схему PostgreSQL, оптимизировал SQL, кешировал горячие read-модели в Redis; покрыл сценарии pytest.

Высоконагруженная страховая платформа (США)

Январь 2022 – Март 2024

Набор использованных технологий: Python, Golang, Flask, Kafka, gRPC, PostgreSQL, MySQL, SQLAlchemy, AWS (EKS, EC2, S3, RDS, IAM, CloudWatch, Lambda), Kubernetes, ArgoCD, Docker, Redis, Prometheus, Grafana, Pytest, Artifactory

Описание проекта

Микросервисная платформа в регулируемой среде: проверка доступности услуг и синхронизация клиентских данных для миллионов пользователей. Критичны надёжность на сезонных пиках, аудит изменений и поставка в AWS/Kubernetes.

Что было сделано

- Разрабатывал и поддерживал мультисервисный пайплайн проверки статусов на Kafka — синхронизация между сервисами.
- Сопровождал плотный gRPC-трафик между соседними микросервисами; обеспечивал устойчивость на сезонных пиках.
- Работал на стыке Python и Golang на критичных участках; участвовал в переносе отдельных стадий пайплайна на Python.

- Эксплуатировал микросервисы в AWS EKS/Kubernetes; деплой через ArgoCD; CI/CD и артефакты поставки (Artifactory).
- Соблюдал практики безопасной работы с чувствительными данными: минимальные права, аудит, наблюдаемость.
- Оптимизировал горячие пути и согласование статусов; применял параллельную обработку под пиковую нагрузку.
- Поддерживал backend API и контракты данных для внутренних React-инструментов операционных команд.
- Автоматизировал data-fix скриптами для исторических несоответствий в БД; участвовал в разборе production-инцидентов.
- Писал автотесты и собирал доказательства для приёмки фич; проводил code review в распределённой команде.

Платформа интеграций доставки для ресторанов

Март 2021 – Декабрь 2021

Набор использованных технологий: Python, FastAPI, asyncio, GCP (Cloud Run, Pub/Sub, Datastore, Cloud Scheduler), Redis, Docker, OpenAPI/Swagger, Sentry, Pytest

Описание проекта

Единый внутренний API и кабинет поверх нескольких внешних площадок доставки. Микросервисы, событийная модель и saga-оркестрация для согласованности заказов и статусов при сбоях на стороне партнёров.

Что было сделано

- Отвечал за интеграции с крупными площадками доставки (Uber Eats, Grubhub, DoorDash, GloriaFood).
- Свёл разные внешние API к единому внутреннему контракту: ретраи, rate limit, устойчивость к смене поведения партнёров.
- Проектировал и развивал микросервисную архитектуру на GCP: Cloud Run, Pub/Sub, Datastore, Cloud Scheduler.
- Реализовал событийные потоки и saga-оркестрацию для согласованности заказов и статусов.
- Перевёл сервисы с Falcon/Starlette на FastAPI; REST-слой с OpenAPI для продукта и адаптеров провайдеров.
- Обеспечил ежедневные высоконагруженные синхронизации и сверку статусов под лимитами внешних API.
- Настроил кеширование в Redis, Docker, Sentry; документировал интеграционные контракты; проводил code review.

Образовательная платформа · платежи и учёт

Январь 2020 – Февраль 2021

Набор использованных технологий: Python, Django, Django REST Framework, Celery, PostgreSQL, Redis, Docker, Docker Compose, Swagger, GitLab CI/CD, Sentry, Pytest

Описание проекта

Агрегатор образовательных услуг с контуром продаж, платежей и учёта: витрина, биллинг, интеграция с бухгалтерией, согласованность проводок и миграция с legacy без потери целостности.

Что было сделано

- Разрабатывал backend на Django/DRF: каталог услуг, карточки клиентов, сценарии покупки.
- Спроектировал модель данных PostgreSQL под услуги, заказы, платежи и учётные статусы.
- Реализовал платёжный и биллинговый контур: статусы оплаты, защита от дублей и гонок на денежных путях.
- Интегрировал бухгалтерскую систему; обеспечил согласованность проводок и auditable-переходы финансовых сущностей.
- Написал и провёл миграционные скрипты переноса данных с контролем целостности и синхронизации.
- Реализовал фоновые задачи на Celery + Redis; идемпотентные webhook/callback платёжных провайдеров.
- Внедрил кеш каталога в Redis, оптимизировал SQL; настроил CI/CD, Docker Compose, Swagger, pytest, Sentry.